Project Process Document

LILA BLACK — Player Journey Visualizer

For internal reference and technical walk-through. May be shared on request.

1. Thinking Process & Approach

When I received the assignment, I broke it into three questions before writing any code:

1. What does the raw data actually look like? (schema, format, coordinate system, encoding quirks)
2. What does a Level Designer actually need? (not just what the spec says, but what would make the tool genuinely useful day-to-day)
3. What is the simplest architecture that delivers all requirements and can be deployed and shared in under 5 days?

> *Key insight: The spec said 'parquet files' but the files had .nakama-0 extensions. Checking the first 4 bytes (PAR1 magic number) revealed they were parquet-formatted — the extension was irrelevant. This kind of data recon is what separates a working tool from a broken one.*

2. Project Workflow

Phase 1 — Learning & Data Recon (Day 1)

- Read the README to understand coordinate system, map origins, scale values
- Inspected first .nakama-0 file with hex dump — found PAR1 magic bytes
- Loaded with pandas, printed columns and first row to understand schema
- Discovered event column is bytes-encoded, not strings
- Discovered bot detection: standard UUID4 = human, non-UUID = bot
- Verified coordinate formula with a known sample position

Phase 2 — First Build: Streamlit Prototype (Day 1 – Day 2)

- Built and deployed a working Streamlit app by Day 2 — it visualized the data correctly but was too slow due to server-side rendering (full Python re-run on every interaction)
- Decided to switch to Next.js 14 + Canvas for instant client-side rendering — no server re-runs on every filter change
- Chose static export + Vercel for zero-cost hosting with no backend server needed
- Decided to split data by map to avoid loading 19 MB upfront

Phase 3 — Stack Switch & Full Rebuild (Day 2 – Day 3)

- Built preprocess_new.py: parquet → 3 JSON files with pixel coords
- Scaffolded Next.js app with TypeScript, Tailwind, component structure
- Built MapCanvas.tsx: canvas renderer with dot mode and heatmap mode
- Built Sidebar.tsx with all filters; custom match dropdown with search
- Built StatsPanel, TimelineSlider, Legend, LoadingOverlay components
- Wired up useMapData hook with in-memory cache for lazy loading

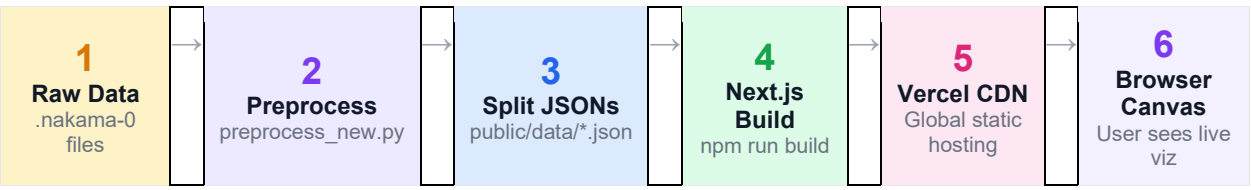
Phase 4 — QA & Bug Fixes (Day 3)

- Bug: preprocess output 0 events — bytes not decoded before set lookup
- Bug: Deaths = 0 — stats filter had wrong event type ('Death' vs 'Killed')
- Bug: minimap not loading — backend not running (separate terminal needed)
- Bug: match IDs truncated in dropdown — replaced native select with custom dropdown
- Added full number display below 10k, abbreviated above for better readability

Phase 5 — Deploy & Submission (Day 3)

- Git push to GitHub public repo
- vercel --prod → live URL in under 2 minutes

3. Architecture Workflow Diagram



Component Architecture

```
page.tsx (state, data loading)
  ├── Sidebar.tsx     (filters: map, date, match, events, heatmap mode)
  ├── MapCanvas.tsx   (canvas: minimap + event dots / heatmap)
  ├── StatsPanel.tsx  (8 live stat cards)
  ├── TimelineSlider.tsx (scrubber + auto-play)
  └── LoadingOverlay.tsx (spinner while JSON loads)
```

4. How the System Works Internally

Data Pipeline

preprocess_new.py runs once, offline. It uses DATA_DIR.rglob('*.nakama-0') to find all 1,243 files recursively across February_10 through February_14 subfolders. For each file:

- pandas.read_parquet() reads the parquet format (despite .nakama-0 extension)
- Column names are lowercased and stripped
- event values decoded: isinstance(v, bytes) → v.decode('utf-8')
- World coords converted: u = (x - ox) / scale, px = u * 1024
- Y-axis flipped: py = (1 - v) * 1024 (game Z is forward, image Y is down)
- Bot detection: uuid.UUID(uid) succeeds for humans, raises ValueError for bots
- seq assigned as sequential integer per match for timeline ordering

Canvas Rendering

MapCanvas.tsx uses a single <canvas> element sized to its container via ResizeObserver. On each render:

- ctx.clearRect → fill black background

- Compute scale = min(W, H) / 1024 and offX/offY to center the minimap
- Draw minimap image at scaled dimensions
- In dot mode: iterate visible events, draw colored arcs at (offX + px*scale, offY + py*scale)
- In heatmap mode: call buildHeatmap() → get ImageData → putImageData()

Heatmap Algorithm

buildHeatmap() in heatmap.ts:

4. Accumulate points into a 256×256 float grid
5. Run separable 1D Gaussian blur (horizontal pass then vertical pass) — O(n×k) vs O(n×k²) for 2D
6. Normalize to 0–1 range
7. Map each grid cell to RGBA using a 4-stop colour gradient (blue→cyan→yellow→red)
8. Up-sample to canvas resolution by nearest-neighbour lookup

5. Future Improvements

Short-term (next sprint)

- Zoom and pan on the canvas — mouse wheel + drag
- Player trail lines — polylines connecting Position events per user per match
- PNG export button — screenshot canvas for sharing in design docs
- Tooltip on hover — show event type, player ID, timestamp on mouse-over

Medium-term

- Side-by-side compare mode — two canvases with independent filters
- Zone annotations — level designers can draw rectangles and label hotspots
- Auto-cluster detection — DBSCAN on kill events to surface high-risk zones automatically
- Live data pipeline — Supabase or Firebase so new match data auto-appears without re-deploy

Long-term

- 3D elevation overlay — use Y coordinates (height) as a third dimension for cliff/high-ground analysis
- Replay with player labels — show player nicknames on their position dots during playback
- Multi-session comparison — overlay data from different days to spot map evolution

6. How AI Was Used

Claude (Anthropic) was used as the primary development assistant throughout the build. AI was not used to blindly generate code — it was used as a pair programmer to solve specific, well-defined problems.

AI Prompt Patterns Used

Phase	Prompt approach used	Output
Data recon	`"What format are files with magic bytes 50 41 52 31?"`	Identified as Parquet despite .nakama-0 extension

Phase	Prompt approach used	Output
Schema detection	`"My pandas df has columns user_id, match_id, map_id, x, y, z, ts, event — the event col returns b'Position'. Fix the preprocess script."`	decode_val() function + bytes handling
Coordinate mapping	`"Verify my pixel formula against the README: px = (x-ox)/scale * 1024"`	Confirmed + Y-flip explanation
Bot detection	`"How do I distinguish human vs bot UUIDs in Python?"`	uuid.UUID() try/except pattern
Canvas rendering	`"Build a React Canvas component that renders 89k dots from a list of {px,py,event} objects using event-specific colors"`	MapCanvas.tsx with offX/offY scaling
Heatmap	`"Implement a client-side Gaussian heatmap from scratch in TypeScript — no libraries"`	heatmap.ts with separable 1D blur
Match dropdown	`"The match IDs are long UUIDs. Build a searchable dropdown that shows full names and wraps them"`	Sidebar.tsx custom dropdown
Debugging	`"preprocess outputs 0 events. First file sample: {event: b'Position', map_id: 'AmbroseValley'}. Fix the filter logic."`	Identified bytes not being decoded before set lookup

What AI Did Well

- Rapidly scaffolding boilerplate (component structure, TypeScript interfaces, CSS variables)
- Debugging based on error messages + sample data — faster than Stack Overflow
- Translating algorithm descriptions ("separable Gaussian blur") into working TypeScript

Where Human Judgment Was Critical

- Identifying that .nakama-0 files were parquet — AI helped confirm, but manual hex inspection was the real signal
- UX decisions: what a Level Designer actually needs vs what the spec literally says
- Debugging the 0-events bug required reading sample output carefully, not just prompting

> *Rule of thumb used: AI for implementation speed, human judgment for architecture decisions and debugging root causes.*

7. Assumptions & Notes

- Dates assumed as 2026-02-XX based on folder names (February_10 etc.) — no explicit year in data
- Bot detection via UUID format — assumes all human players have standard UUID4 identifiers from Nakama auth
- Coordinate system Y-flip assumed from README spec: game Z (depth) maps to image Y (vertical), with inversion
- match_id derived from filename stem (UUID portion before .nakama-0) — cleaner than the raw match_id column which included the extension
- GrandRift minimap uses .png (not .jpg as originally listed in plan) — confirmed from actual files
- Timeline seq is per-match integer index based on ts_ms ordering — enables smooth playback without real timestamp complexity